Practical No.: 11, 13, 14

Title: Create a Node.js Application to Store / Display / Update / Delete Student Information in a Database

Aim:

To develop a **Node.js application** that stores student information in a **MongoDB database** using **Express.js** and **Mongoose**.

Theory:

What is Node.js?

- **Node.js** is a JavaScript runtime that allows building scalable **server-side applications**.
- It is commonly used to develop **RESTful APIs** and **database-driven applications**.

What is MongoDB?

- **MongoDB** is a **NoSQL database** that stores data in **JSON-like format**.
- It is used for **fast and scalable** data storage.

What is Mongoose?

- **Mongoose** is an **Object Data Modeling (ODM)** library for **MongoDB and Node.js**.
- It provides an easy way to interact with the database using JavaScript objects.

Steps to Implement the Application:

1. Install **Node.js**, **MongoDB**, and **Postman**.
2. Set up a **Node.js project** and install required dependencies.
3. Create a **MongoDB database** to store student details.
4. Develop a **REST API** to perform **CRUD (Create, Read, Update, Delete) operations** on student data.
5. Test the API using **Postman**.

Code Implementation:

Step 1: Create a New Node.js Project and Install Dependencies

Run the following commands in the terminal:

```
mkdir student-management
cd student-management
npm init -y
npm install express mongoose cors body-parser
```

Step 2: Create a MongoDB Database

- If MongoDB is installed locally, start the database using

```
      mongod --dbpath /path/to/database
```

Step 3: Create server.js File

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

const app = express();
app.use(cors());
app.use(express.json());

// Connect to MongoDB (Change the connection string if using MongoDB Atlas)
mongoose.connect("mongodb://127.0.0.1:27017/studentsDB", {
    useNewUrlParser: true,
    useUnifiedTopology: true
}).then(() => console.log("Connected to MongoDB"))
  .catch(err => console.error("MongoDB Connection Error:", err));

// Define Student Schema
const studentSchema = new mongoose.Schema({
    name: String,
    course: String,
    age: Number
});

const Student = mongoose.model("Student", studentSchema);

// API Routes

// GET: Fetch all students
app.get("/students", async (req, res) => {
    const students = await Student.find();
    res.json(students);
});

// POST: Add a new student
app.post("/students", async (req, res) => {
    const newStudent = new Student(req.body);
    await newStudent.save();
    res.status(201).json(newStudent);
});

// PUT: Update student details
app.put("/students/:id", async (req, res) => {
    const updatedStudent = await Student.findByIdAndUpdate(req.params.id, req.body, {
new: true });
    res.json(updatedStudent);
});
// DELETE: Remove a student
app.delete("/students/:id", async (req, res) => {
    await Student.findByIdAndDelete(req.params.id);
    res.send("Student deleted");
});
// Start Server
```

```
app.listen(5000, () => console.log("Server running on port 5000"));
```

Step 4: Run the Server

node server.js

Step 5: Test the API using Postman or a Frontend

Method	Endpoint	Description	Example Data
GET	`/students`	Fetch all students	-
POST	`/students`	Add a new student	`{ "name": "John", "course": "BCA", "age": 20 }`
PUT	`/students/:id`	Update student details	`{ "course": "MCA" }`
DELETE	`/students/:id`	Remove a student	-